

Selección de índices en PostgreSQL

Pre-entrega 6 · SQL y PostgreSQL · Carolina Machado

Se elige un índice según el operador de búsqueda, el tamaño real de la tabla y el plan de ejecución. Los ejemplos son propuestas técnicas; sin mediciones sobre la carga de trabajo no se prometen mejoras de tiempo.

Consulta A · tracking_id = valor exacto

Un índice Hash sirve para igualdad exacta. Si tracking_id debe ser único, conviene declarar UNIQUE, que ya crea un B-Tree y resuelve la búsqueda exacta sin mantener un segundo índice Hash. Por eso, en un diseño real empezaría por UNIQUE y mediría antes de considerar otra estructura.

```
CREATE UNIQUE INDEX idx_pedidos_tracking
ON pedidos USING btree (tracking_id);
```

Consulta B · rango de fecha y orden por monto

Un B-Tree sobre fecha_venta permite localizar un rango. El orden por monto puede requerir un paso de ordenamiento: un índice (fecha_venta, monto) no garantiza que todas las filas del rango queden globalmente ordenadas por monto, porque primero ordena por fecha. Se comprueba con EXPLAIN (ANALYZE, BUFFERS).

```
CREATE INDEX idx_ventas_fecha
ON ventas USING btree (fecha_venta);

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM ventas
WHERE fecha_venta BETWEEN DATE '2026-01-01'
                        AND DATE '2026-01-31'
ORDER BY monto;
```

Consulta C · propiedades JSONB

Un índice GIN descompone el documento para búsquedas de contención. La consulta usa el operador @>, compatible con el índice GIN por defecto sobre JSONB.

```
CREATE INDEX idx_productos_propiedades
ON productos USING gin (propiedades);

SELECT * FROM productos
WHERE propiedades @> '{"color":"rojo","talla":"L"}'::jsonb;
```

Costo de mantenimiento

Con 15 índices en inventario, cada INSERT, UPDATE o DELETE debe mantener las entradas afectadas. Aumentan el espacio y el trabajo de escritura; no significa que una inserción sea exactamente 16 veces más lenta. Se retienen índices justificados por consultas frecuentes y se miden lecturas, escrituras y tamaño.

Búsqueda con UPPER(nombre)

Un B-Tree normal en nombre generalmente no sirve para comparar UPPER(nombre) porque el predicado opera sobre una expresión. Si esa búsqueda es recurrente, se crea un índice de expresión y se usa el mismo predicado en la consulta.

```
CREATE INDEX idx_clientes_nombre_upper
ON clientes (UPPER(nombre));

SELECT * FROM clientes WHERE UPPER(nombre) = 'JUAN';
```